PRODUCT AND ENGINEERING REFERENCE

Planning & Design Notes

A desktop-first HR workspace for employee records, annual base salaries, compensation insights and traceable salary reviews.

21 / 29	7	10,000
Reporting features completed	Application database tables	Synthetic employee seed capacity

Purpose

Replace scattered compensation spreadsheets with a searchable directory, validated salary updates, immutable application-level history and currency-specific summaries. Give HR useful review indicators without automatically deciding that a salary is wrong or unfair.

Audience and Permissions

The current persona is one seeded HR superadmin. This user can view dashboards, create employees, update salaries with confirmation, read salary history and investigate data-quality findings. There is no employee self-service, public registration or separate read-only HR role.

Current Engineering Checkpoint

Stack	Ruby 3.4.9, Rails 8.1.4, PostgreSQL, React 19, TypeScript and Vite.
Latest recorded checks	358 Rails specs and 43 frontend tests passed; frontend production build, Ruby lint and Brakeman scan passed in prior local verification.
Schema snapshot	Version `20261013000004`; 72 columns, eight foreign keys and 19 secondary indexes.
Release state	Latest committed checkpoint is `88c4571`, single-HR authentication. Reporting, data-quality and status-page enhancements remain local and uncommitted at this snapshot.

> **Reading this document:** implemented behavior describes the current repository, not proof of deployment. Proposed work is explicitly labelled. Test counts are the latest recorded local results, not a new test run for this document.

No credentials, tokens, private employee records or production database contents are included. Completion uses the agreed currency-specific scope; it is not a legal, pay-fairness or production-performance certification.

Scope and Success Criteria

The original feature list contains 30 items. The common-currency India/US comparison is excluded, leaving 29 tracked items. Normalized salary averages are replaced by per-currency averages in the agreed scope.

Area	Delivered Behavior	Acceptance Focus
Employee directory	Search name/ID; combined country, department and status filters; salary sorting; server pagination.	Accurate counts, stable order, bounded pages and no accidental currency conversion.
Employee creation	Validated identity, role, geography, status and initial compensation.	Reject duplicate IDs and invalid details; show the new employee and refresh summaries.
Salary management	Current salary details, edit/review/confirm flow and old/new history.	No write before explicit confirmation; salary and history succeed or fail together.
Organization insights	Headcount, annual/monthly estimates, average, median, ranges, country/department rankings and salary bands.	Exact backend calculations, currency-separated money and reconciled headcounts.
Salary analysis	Comparable role groups, high-salary outliers and below-department-average lists.	Full filtered cohort before pagination; transparent peer thresholds.
Data quality	Eight manual checks, saved evidence, HR decisions, audit history and rescans.	No automatic salary correction; decisions persist; changed sources require review again.
Access and status	Single-HR login, logout and expiry; public status page with light/black theme and portal link.	No protected data before session verification; no API endpoint list on the public page.

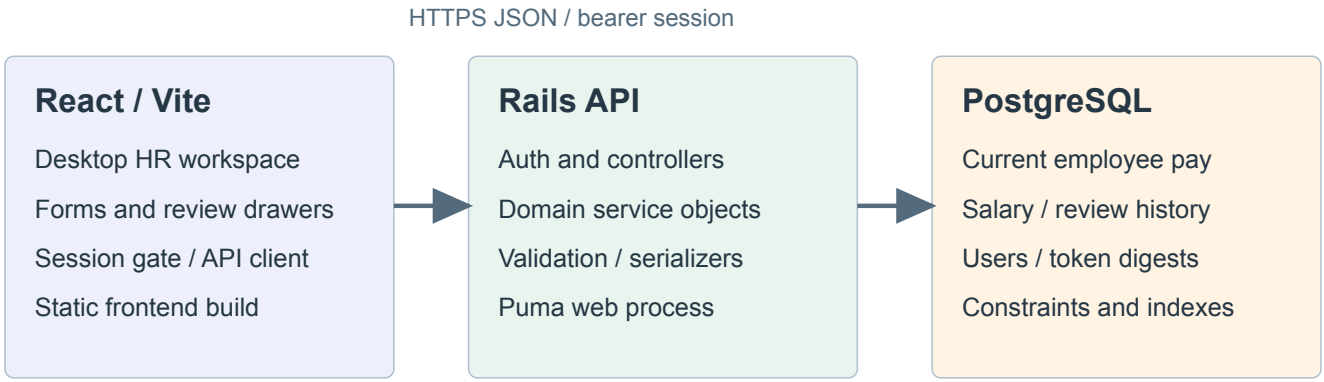
Current Completion

21 completed, eight not fully completed (about 72%). Within the adjusted scope, 17 P0 items and four P1 items are implemented. Two P1 areas are partial; six P2 features remain unimplemented. The feature-coverage PDF contains item-by-item traceability.

Deliberate Boundaries

- Desktop web testing only for now; mobile validation is deferred.
- No FX conversion, payroll execution, tax/benefit calculation, bonus accounting or future salary scheduling.
- Country/role fairness with contextual evidence is deferred by request. No seniority, experience, hours or performance fields were added.
- History is an audit trail, not a salary-history editing surface. Statistical flags do not establish unfair pay.

Architecture Design



Boundary	Responsibilities
`Frontend/`	AuthGate, API client, directory, insights, salary analysis, history and data-quality views. UI preferences are local; the API remains authoritative.
`Backend/`	JSON API under `/api/v1`. Controllers handle auth, parameters and HTTP responses; models enforce record rules; services own updates and reporting.
Domain services	EmployeeDirectory, SalaryUpdater, CompensationInsights, SalaryAnalysis, SalaryDataQualityChecker/Rules and SalaryDataAnomalyReviewer.
Database	Seven primary application tables. PostgreSQL performs aggregates and cohort calculations, enforces integrity and supports transactional locks.

Design Rationale

- Keep the browser focused on interaction, not duplicating backend financial or review rules.
- Use service objects for atomic writes and complex reads without introducing a separate service or distributed transaction.
- Separate static frontend and API deployments; retain one repository and independent CI checks for each.
- Keep the public Rails root separate from the HR application. It reports liveness, not database readiness, and contains no endpoint documentation.

Workflows and UI Notes

Navigation and Information Hierarchy

The signed-in workspace exposes Overview, Employees, Compensation, Salary history, Salary analysis and Data quality. Tables prioritize scanning and comparison. Search, filters, currency selectors, pagination and focused drawers keep the primary work in context.

Journey	Interaction Sequence	Protection / Outcome
Sign in	Email/password submission; issue session; verify saved sessions before restoring the workspace.	Invalid credentials show an error; no employee requests while signed out or unverified.
Find an employee	Search name or business ID; combine filters; sort salary; move through pages; open employee details.	Server-side matching and bounded result pages; deterministic ordering.
Create employee	Open New employee; enter required identity and salary fields; submit; inspect validation feedback.	Duplicate-ID errors preserve the draft. Success searches for the created ID and refreshes insights.
Change salary	Open salary drawer; edit amount/currency/date; review old/new values; explicitly confirm.	Validated atomic write; append history; close drawer and refresh relevant views.
Review a flag	Run checks; inspect observed/reference values and snapshot; choose decision; supply notes; save.	Stale sources reject review. Open employee salary for correction, then rescan to clear the flag.
Sign out / expire	Revoke the current session or handle expiry/401; clear browser session and protected workspace.	Salary data is no longer visible; sign-in is required again.

Implemented Interaction States

- Loading, empty, validation-error, unavailable and retry states; disabled controls during writes.
- Salary confirmation separate from editing; feedback after saves; filters and page counts remain visible.
- Abortable reads and response keys avoid displaying an earlier report under a new mode/filter.
- Dialog focus handling, Escape dismissal, labelled controls and icon tooltips. Drawers switch rather than nest when opening employee salary from a finding.
- Workspace dark/light/system preferences; public backend page follows native appearance until an explicit light/black preference is saved.

Proposed Usability Validation

Observe HR completing real tasks, check terminology and currency labels, audit keyboard/screen-reader behavior and verify long names/large values. Current browser tests are not a complete accessibility audit. Mobile acceptance remains outside the current test plan.

Data Model Decisions

Table	Purpose and Important Relationships
`employees`	Current directory and annual compensation. Unique string business employee_id; numeric primary id for API lookups and foreign keys.
`salary_changes`	Old/new amounts, currencies, effective dates and change timestamp; belongs to an employee.
`users`	HR credentials and activity status. Separate from employee payroll records; no user-to-employee link is implemented.
`sessions`	Token digest and expiry; belongs to an HR user.
`salary_data_quality_scans`	Run timing, requesting user and summary counters.
`salary_data_anomalies`	Fingerprint, rule, severity and JSONB evidence; employee and latest detecting scan; optional latest reviewer.
`salary_data_anomaly_reviews`	Decision, notes, author and time; belongs to a finding and HR user.

Identity, Precision and Integrity

- Seven bigint primary keys, eight foreign keys, 19 secondary indexes and nine CHECK constraints. Four secondary indexes are unique.
- `employees.employee_id` is a business string, while salary/finding `employee_id` references `employees.id`. Do not interchange these identifiers.
- Salary columns use `numeric(18,2)`. API amounts are fixed two-decimal strings; server calculations round only reported values.
- Database constraints enforce supported currencies, employee status, nonnegative pay, review statuses and superadmin role. Required strings also receive Rails nonblank validation.
- Country, department and designation are strings, not normalized reference tables. JSONB evidence is required, but its shape is enforced by application construction rather than a SQL JSON schema.

History and Deletion Policy

Salary changes and review events are read-only once persisted through Rails. This is not a database-level immutable ledger: direct SQL can bypass the guard. Foreign keys do not cascade-delete audit records. Model associations restrict deleting employees/findings with history. Salary history does not currently store the editing HR user's ID.

Supporting Reference

EmployeePortal-Database-Schema.pdf contains the ERD and every column, default, index and constraint. Its JSON/SVG/Mermaid sources are stored alongside this document. The schema describes local development; production migration state must be checked separately.

API Design Notes

JSON resources are grouped under `/api/v1`. Protected operations require an active HR superadmin bearer session and use `Cache-Control: no-store`. This internal document is not served on the public backend status page.

Operation	Contract / Purpose
`POST /session` `GET / DELETE /session`	Login accepts a session email/password object. Return/verify the user and expiry; revoke the current session on logout.
`GET /employees` `POST /employees`	Paginated search/filter/sort; validated employee creation. Collection reads return data plus page/count metadata.
`GET /employees/:id`	Read identity and compensation using numeric database ID.
`GET /employees/:id/salary` `PATCH / PUT` same path	Read or update base_salary, currency and effective_date. Updates require `confirmed: true` in the salary object.
`GET /employees/:id/salary_history`	Read old/new compensation and change timestamps; no history-edit endpoint.
`GET /compensation_insights` `GET /salary_analysis`	Currency-separated summaries and paginated reports selected by report parameter.
`POST /salary_data_quality_scans`	Run synchronous manual checks across the database; return completed scan counters.
`GET /salary_data_anomalies` `GET / PATCH / PUT /salary_data_anomalies/:id`	Filtered queue, evidence and paginated review history; append HR decisions without writing salaries.

All paths in the table are relative to /api/v1. Nested salary routes use employee_id as the route parameter name, but its value is still the numeric employees.id.

Query and Error Rules

Country, department and status combine; directory search uses case-insensitive name/business-ID matching. Directory and analysis pagination defaults to page 1 / 25 rows, capped at 100; positive integers are required. Stable ties use employee ID or the report's group names.

Typical status codes: `200` read/update; `201` login/create/scan; `204` logout; `400` malformed required object; `401` invalid session; `403` wrong role; `404` unknown record; `409` stale finding review; `422` validation; `429` login limit.

Salary Transaction Boundary

SalaryUpdater locks the employee, captures existing values, validates the requested values, saves only when changed, then writes history within the same transaction. Failed validation/history insertion rolls back the update. Identical values create no change row. A future effective date is stored immediately; there is no deferred activation scheduler.

Reporting Design and Boundaries

Metric / Report	Definition	Boundary
Annual expenditure	Sum of current stored annual base salary per currency.	No cross-currency total; not a historical payroll ledger.
Monthly estimate	Annual base total divided by 12 before rounding.	No tax, benefit, bonus, prorating or scheduled payroll treatment.
Average / median	Arithmetic average; median averages the middle values when headcount is even.	Decimal server calculations; zero salary participates.
Ranges / bands	Minimum/maximum; four fixed salary ranges with inclusive lower and exclusive upper bounds.	Reporting bands are currency-specific, not pay policy.
Rankings / counts	Country/department totals or averages within a currency; consolidated country headcount across currencies.	Money stays separate even when headcounts are combined.
Role comparisons	Same country, designation and currency; count, average, median, range and salary gap.	No experience, seniority, working-hour or performance adjustment.
High-salary outliers	At least five peers including the employee; salary strictly above Q3 + 1.5 x IQR using discrete quartiles.	If Q1 = Q3, threshold is 1.5 x Q3. A flag is not a finding of unfair pay.
Below average	Salary strictly below the unrounded department/currency average.	No automatic raise recommendation or fairness conclusion.

Currency and Salary-Band Policy

Supported currencies: INR, USD, GBP, EUR and JPY. Band boundaries are INR 500,000 / 1,000,000 / 2,000,000; USD, GBP and EUR 50,000 / 100,000 / 150,000; JPY 3,000,000 / 6,000,000 / 9,000,000. Zero-count bands remain visible and the last band is open-ended.

Scope Before Pagination

Insight filters apply to every statistic. Salary-analysis filters define the entire comparison population before paging employee/group results. Default scopes include active and inactive employees; choose active explicitly when estimating active-staff base pay.

Query Design

CompensationInsights uses PostgreSQL grouping sets and filtered band counts rather than loading every employee into Ruby. SalaryAnalysis computes cohort statistics in SQL and loads only a requested page. Rankings in the UI use the returned currency-specific aggregates.

> **Interpretation:** currency-specific reports deliberately avoid fabricated FX conversion. Current salary plus history is not enough to reconstruct certified year-over-year payroll expenditure without additional rules for employee populations and timing.

Checks and Review Lifecycle

Check	Implemented Trigger
Active zero pay	Active employee with base_salary = 0. Inactive zero pay is not flagged by this rule.
Extreme peer ratio	Positive salary at least 10 x, or at most 0.1 x, positive median; same country/role/currency; minimum five employees.
Annual/monthly pattern	Latest same-currency change near 12 x or 1/12 x a positive old salary, within 2% tolerance.
Large salary change	Latest same-currency change increases at least fivefold or falls at least 80%; annual/monthly pattern takes precedence.
Same amount, new currency	Currency switched while an identical positive numeric amount remained; information flag, not assumed conversion error.
Distant future date	Current effective date more than 365 days after scan date; shorter future dates remain valid.
History mismatch	Current amount/currency/effective date differs from latest history, ordered by changed_at then ID descending.
Blank details	Blank business ID, name, department, country or designation, potentially after an import bypassing Rails validation.

Scan Design

Manual synchronous scan, 500 employees per batch, one grouped peer-statistics query and a lateral latest-history lookup. A PostgreSQL transaction advisory lock serializes full scans. Scan and findings roll back on failure. Fingerprints reuse unchanged evidence and preserve prior decisions.

Decision State Is Separate From Finding Activity

Pending, Investigating, Confirmed error and Legitimate are HR decisions. Active/Cleared is detection state. Confirmed error and Legitimate require notes, capped at 2,000 characters. Every saved decision appends reviewer/time history and updates the finding's latest decision.

Source changes or cleared findings reject review with 409 after locked revalidation. A rescan clears superseded/no-longer-detected evidence without deleting its reviews. Changed evidence creates a new Pending finding. Corrections use the normal confirmed salary-update flow; scans never edit salaries.

Review Queue and Limits

Default UI is active Pending findings. Currency/employee/decision filters, pagination and Include cleared are available. Filters narrow the queue, not scan baselines; peers include active and inactive employees. Counters cover all employees and results are as of the last check, not continuous monitoring.

Access and Protection Notes

Implemented Control	Design / Limit
Authentication	Bcrypt password hashes, active-user checks and opaque random bearer tokens. PostgreSQL stores SHA-256 token digests, not raw tokens.
Session lifecycle	Eight-hour sessions; logout revokes the current token; password/activity changes revoke user sessions. Browser verifies a saved session before showing protected views.
Browser storage	Per-tab sessionStorage, with in-memory fallback. This limits persistence but is still accessible to same-origin JavaScript; it is not an HttpOnly cookie.
Authorization	Only the superadmin role is supported; server-side checks protect employee, insight and review routes. Hiding a button or route list is not authorization.
Abuse resistance	Login allows 10 attempts per IP per three minutes through a process-local MemoryStore. This is not a shared global limit across multiple Rails processes.
Browser origin policy	Restricted frontend origin for /api/* and explicit methods including OPTIONS. CORS permits a browser origin; it does not authenticate an API caller.
Public backend page	Application liveness, time and portal link only; no route list/version/base path. CSP restricts scripts to self and prevents framing; no employee queries for status.
Secret handling	Backend environment for Rails key, DB connection and HR provisioning password; parameter-log filtering. No credentials in VITE_ values, screenshots or PDFs.

Proposed Production Hardening

- Review frontend CSP and dependencies, and avoid untrusted scripts because browser bearer storage remains vulnerable to same-origin script compromise.
- Use a shared login-rate-limit store before scaling to multiple processes; assess MFA or managed identity before adding broader access.
- Rotate any exposed secrets; apply least-privilege database access; define backup/restore, retention and salary-data incident procedures.
- Consider database-level audit protections and adding the salary-changing actor if stronger traceability is required.

No claim is made that an independent penetration test, comprehensive security audit or production readiness certification has been completed.

Testing and Nonfunctional Plan

Existing Verification	Coverage / Boundary
358 Rails examples	Model constraints, API behavior, auth, financial calculations, atomic salary/history writes, seed idempotency, anomaly lifecycle and deployment script.
43 frontend tests	Five API-client tests and 38 desktop browser tests, including five backend-theme fixtures. Workflow tests intercept API responses.
10,000-record checks	Seed generation and bounded anomaly batches tested; real local workspace contains 10,032 employees. No production load guarantee is implied.
Build and CI	TypeScript/Vite build, RuboCop, Brakeman, gem audit job, Rails specs against PostgreSQL 16, and Linux Chromium browser checks.
Local smoke checks	Real Rails/UI data, desktop widths 1280/1440/1920 and health/theme behavior. One real local scan checked 10,032 employees in about 339 ms.

Performance Decisions Already Implemented

- Server pagination with a 100-row API cap and indexed salary/filter/foreign-key paths; 300 ms debounced directory search.
- SQL aggregation for insights and peers; latest history loaded in batched queries, not per employee.
- Explicit manual scans and disabled duplicate actions in the UI. Scans still run inside a web request and may become expensive on larger data.

Proposed Acceptance Before a Production Release

1. Run the full test/build/security pipeline on the exact release commit and add a real staging login-to-salary-change-to-history flow.
2. Benchmark warm and cold start behavior, p50/p95 response times, concurrent browsing/writes and scans on deployment-sized hardware. Agree response-time targets from those results.
3. Profile indexed queries and ILIKE search before adding indexes or caching. Introduce a background scan with progress only if request timeouts/resource pressure warrant it.
4. Test failed migrations, backups/restores, network interruption and stale concurrent salary editing; verify accessibility and long-content layout.

> **Precision risk to address:** backend amounts retain decimals as strings, but the UI money formatter currently converts them to JavaScript Number. Very large values near the database's numeric limit may lose cents in display; add a safe display-range rule or decimal-aware formatting before using such values.

Deployment Design Notes

This describes repository configuration and a proposed release procedure, not a fresh verification of hosting-provider settings or current live deployment.

Component	Repository Configuration
Rails on Render	Root Backend; build `./bin/render-build.sh`; start Puma with config/puma.rb bound to 0.0.0.0:$PORT; liveness path /up.
PostgreSQL	Production primary uses DATABASE_URL. The configuration also declares cache/queue/cable databases; deployment script explicitly migrates primary.
Frontend static build	Frontend root, npm ci / npm run build; serve dist. VITE_API_BASE_URL includes the backend /api/v1 prefix and is embedded at build time.
Backend secrets	RAILS_ENV=production, configured Ruby version, DATABASE_URL and RAILS_MASTER_KEY. Values belong in host settings, not the frontend or repository.
HR provisioning	HR_LOGIN_EMAIL, strong HR_LOGIN_PASSWORD, SEED_HR_USER=true and build migrations enabled for one-time provisioning. Remove the seed flag afterward.
Synthetic employees	Only explicitly enabled using ALLOW_PRODUCTION_SEEDS and SEED_EMPLOYEE_COUNT. Remove those flags after a demo seed; do not treat demo records as real HR data.

Browser Connectivity

The current configured frontend origin is `https://employee-portal-frontend-1hdp.onrender.com`; the backend origin is `https://employeeportal-z31o.onrender.com`. Rails CORS allows the frontend origin for /api/*. A new Vercel/preview/custom hostname is not automatically allowed. Changing FRONTEND_URL changes the status-page link, not CORS.

Proposed Release Sequence

1. Review and commit the intended changes; run CI and record the exact release hash. Review backups before applying schema changes.
2. Apply backward-compatible primary migrations, including 20261013000004 for anomaly reviews, before code needs the new tables. Verify migration status in the hosted database.
3. Deploy Rails and build/deploy the frontend with the matching API contract and correct API base URL. Verify CORS preflights and HTTPS access.
4. Provision the single HR account if absent; seed synthetic employees only for an approved demo. Existing passwords are preserved unless explicit rotation is enabled.
5. Smoke-test sign-in, directory, confirmed salary/history, insights, anomaly scans/reviews, expiry/sign-out and public health. Then remove one-time seed/rotation flags.
6. Monitor failures and latency; keep a known-good application release and compatible schema. A database rollback is not assumed safe after salary/audit writes.

/up confirms Rails liveness, not database readiness. Host pricing, plan capabilities, current live state and backup service configuration must be checked separately before release.

Next Work and Decisions

Remaining Feature	Current State / Design Needed
Country/role fairness	Partial basic groups; contextual justification is deferred by request. No new context fields are planned without renewed approval.
Department dispersion	Partial ranges and bands; no variance, standard deviation, department IQR ranking or dedicated spread comparison.
Organization raise simulation	Not implemented. Proposed read-only scenarios with incremental annual/monthly cost per currency.
Department raise simulation	Not implemented. Proposed selected-department percentage scenarios without editing employee salaries.
Year-over-year expenditure	Not implemented. Define historical snapshots, joins/leavers and effective-date treatment before calculating.
Salary-review schedule	Not implemented. Needs a distinct review date/policy and upcoming/overdue lifecycle.
Upcoming increment budgets	Not implemented. Needs planned changes and effective dates with currency-specific forecast rules.
CSV / Excel export	Not implemented. Proposed authorized filtered exports, explicit currency columns and formula-injection safeguards.

Proposed Delivery Order, Not a Committed Schedule

Phase 1: release hardening, staging acceptance, safe large-value formatting, backup/restore verification and deployment observability. **Phase 2:** choose dispersion or read-only simulations as the next product increment. **Phase 3:** scheduling, forecast inputs, historical reporting and protected exports after their data contracts are agreed.

Risk Register and Open Choices

- **Concurrent edits:** row locks preserve atomic history, but there is no optimistic version check for an old browser salary form. Add stale-edit detection before multi-user expansion.
- **Audit strength:** history lacks the salary-changing actor and immutability is application-level. Decide whether stronger attribution/database controls are required.
- **Scale:** scans are synchronous and login limits process-local. Revisit background jobs, query plans and a shared limit store as workload grows.
- **Business semantics:** agree active-only defaults, reference-value governance, effective-date scheduling and anomaly thresholds before interpreting reports as payroll or policy.

Source and Companion Documents

Sources: README.md; Backend/docs/p0_contract.md; database schema/migrations; API controllers and domain services; Frontend/src; Playwright configuration; .github/workflows/ci.yml; Render build script. Companion PDFs: Feature Coverage and Database Schema / ERD. Proposed items in this document do not modify the application.